# Part 1 – Entity Design (Pen & Paper)

Before writing any SQL, you need to understand the business.

## The Business Requirements

QuickBite works like this:

- **Restaurants** sign up to the platform. Each restaurant has a name, address, city, cuisine type (e.g., Italian, Japanese, Burgers), a rating (1–5), and whether they're currently active.
- Each restaurant has a **menu** with items. Each item has a name, description, price, and a category (e.g., Main, Side, Drink, Dessert).
- **Customers** register with their name, email, phone number, city, and registration date.
- Customers place **orders** from a restaurant. Each order has a date/time, a status (`pending`, `preparing`, `delivering`, `delivered`, `cancelled`), and a delivery address.
- Each order contains one or more **order items** — a reference to a menu item and a quantity.
- After an order is delivered, the customer can leave a **review** for the restaurant — a rating (1–5) and an optional text comment.

## Your Task

1. **Identify all the entities** (tables) you'll need.
2. For each entity, list the **columns** and their types.
3. Identify the **relationships** between entities and their types:
   - Which are **one-to-many**?
   - Which are **many-to-many**? (Do you need a relationship table?)
4. Decide where you need **foreign keys**

## Resturants

id: int PK

name: varchar(100)

address: varchar(256)

city: varchar(100)

cuisine_type: varchar(100)

rating: int (1-5)

is_active: boolean

## menu

id: int PK

name: varchar(100)

description: text

Price: decimal (8,2)

category: varchar(100)

resturant_id: int FK(id)

## customers

id: int PK

name: varchar(100)

email: varchar(256)

Phone_number: varchar(20)

city: varchar(100)

registraion_date: date

## orders

id: int PK

time: datetime

status: varchar(20)

delivery_address: varchar(256)

resturant_id: int FK(id)

customer_id: int FK(id)

## order_items

menu_id: int FK(id)

order_id: int FK(id)

quantity: int

## customer_review

id: int PK

rating: int (1,5)

text_comment: text default NULL

order_id: int FK(id)

customer_id: int FK(id)

## Relationships

- Resturants - Menu: one to one
  - each resturant has one menu
  - each menu belongs to one resturant

- Resturants - orders: one to many
  - each resturant receivs many orders
  - each order belongs to one resturant

- customers - orders: one to many
  - one customer can Places many orders
  - each order belongs to one customer

- customer - customer review: one to many
  - each customer can write many reviews
  - each review belongs to one customer

- order - customer review: one to many
  - each order can have many reviews
  - each review belongs to one order

- orders - menu: many to many
  - one order can contain many menu items
  - one menu item can be in many orders